

특별편 2.3 - RAM의 실제 구조

남궁명수

[1] 실제 RAM의 전체 구조

일반적인 x86-64 Linux에서 RAM은 대략 다음과 같이 사용됩니다.

물리 RAM

커널 코드·커널 전역 데이터
커널 자료구조 task_struct, mm_struct, VMA, Page Table 등
Slab/SLUB inode, dentry, socket, 파일 객체 등
Page Cache 최근 읽거나 쓴 파일 데이터
프로세스의 익명 페이지 Heap, Stack, mmap 익명 메모리
프로세스의 파일 기반 페이지 실행 파일의 Code, 공유 라이브러리, mmap 파일
네트워크 Buffer, 장치 Driver Buffer
사용 가능한 Free Page Frame

하지만 실제로는 위처럼 용도별로 깔끔하게 연속 배치되는 것이 아닙니다.

RAM은 다음처럼 여러 페이지 프레임이 섞여 있습니다.

[물리 RAM의 실제 모습]

Frame 0 → Kernel
Frame 1 → Page Table
Frame 2 → Process A Heap
Frame 3 → Page Cache
Frame 4 → Free
Frame 5 → Process B Stack
Frame 6 → Slab
Frame 7 → Process A Code
Frame 8 → Network Buffer
Frame 9 → Process B Heap
Frame 10 → Page Cache
Frame 11 → Free
...

[2] RAM의 기본 관리 단위: 페이지 프레임

일반적으로 Linux는 물리 RAM을 4KiB 크기의 페이지 프레임으로 나누어 관리합니다.

예를 들어, RAM이 16GiB라면

$$16\text{GiB} \div 4\text{KiB} = 4,194,304$$

$$1\text{ GiB} = 2^{30}\text{ Byte}$$

$$1\text{ KiB} = 2^{10}\text{ Byte}$$

약 419만 개의 페이지 프레임으로 나누어 관리하는 셈입니다.

[16GiB RAM]

Frame 0	Frame 1	Frame 2	Frame 3	...
4KiB	4KiB	4KiB	4KiB	

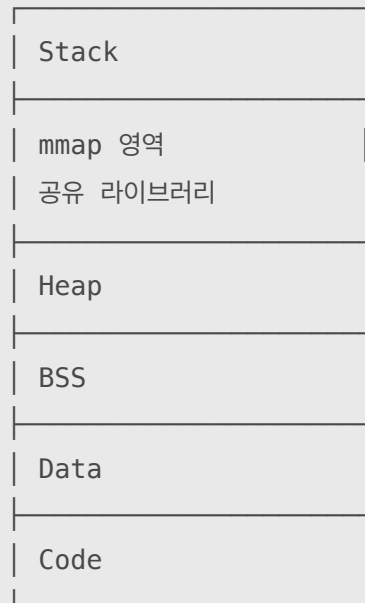
- 가상 메모리의 4KiB 단위: 가상 페이지
- 물리 RAM의 4KiB 단위: 페이지 프레임

[3] 프로세스가 보는 구조와 실제 RAM의 차이

프로세스 A는 자기 메모리가 연속적으로 존재한다고 생각합니다.

[프로세스 A의 가상 주소 공간]

높은 주소



낮은 주소

그러나 실제 RAM에서는 다음처럼 흩어져 있을 수 있습니다.

프로세스 A의 가상 페이지 실제 RAM 페이지 프레임



이 연결 정보를 저장하는 구조가 페이지 테이블입니다.

```
flowchart LR
    VA["가상 주소"]
    VPN["가상 페이지 번호"]
    PT["페이지 테이블"]
    PFN["물리 페이지 프레임 번호"]
    RAM["실제 RAM"]

    VA --> VPN
    VPN --> PT
    PT --> PFN
    PFN --> RAM
```

CPU의 MMU가 페이지 테이블을 이용해 가상 주소를 물리 주소로 변환합니다.

[4] 실제 주소 변환 예시

페이지 크기가 4KiB라고 가정해보겠습니다.
CPU가 다음 가상 주소에 접근합니다.

가상 주소: 0x7FFF1234

주소는 두 부분으로 나뉩니다.

가상 페이지 번호	페이지 Offset
VPN	

페이지 테이블을 확인했더니 해당 가상 페이지가 물리 프레임 300번과 연결되어 있다고 가정합니다.

가상 페이지 0x7FFF1

↓
▼
페이지 테이블

↓
▼
물리 프레임 300

최종 물리 주소는 다음과 같이 만들어집니다.

물리 프레임 시작 주소 + 페이지 내부 Offset

페이지 내부 위치는 바뀌지 않고, 페이지 번호만 가상 페이지 번호에서 물리 프레임 번호로 변환됩니다.

[5] 프로세스의 메모리가 RAM에 들어가는 과정

프로그램을 실행했다고 해서 코드, 힙, 스택 전체가 즉시 RAM에 올라오는 것은 아닙니다.

flowchart TD

```
EXE["실행 파일"]  
VMA["가상 주소 영역 생성"]  
ACCESS["CPU가 주소 접근"]  
FAULT["Page Fault"]  
FRAME["물리 프레임 확보"]  
MAP["페이지 테이블 연결"]  
RUN["명령 또는 데이터 사용"]
```

```
EXE --> VMA  
VMA --> ACCESS  
ACCESS --> FAULT  
FAULT --> FRAME  
FRAME --> MAP  
MAP --> RUN
```

예를 들어, malloc(1GiB)를 호출해도 즉시 RAM 1GiB가 전부 사용되는 것은 아닙니다.

```
char *buffer = malloc(1024 * 1024 * 1024);
```

우선 가상 주소 범위를 확보합니다.

이후 실제로 각 위치에 값을 쓰면 Page Fault가 발생하고, 커널이 물리 페이지 프레임을 하나씩 연결합니다.

```
malloc(1GiB)  
↓  
가상 주소 공간 1GiB 확보  
↓  
buffer[0]에 값 쓰기  
↓  
Page Fault  
↓  
RAM의 4KiB 프레임 할당  
↓  
페이지 테이블에 연결
```

[6] RAM에서 중요한 데이터 종류

[프로세스의 익명 메모리]

원본 파일이 없는 메모리입니다.

- Heap
- Stack
- 익명 mmap
- 프로그램 실행 중 변경된 Private Page

RAM이 부족하면 일반적으로 Swap으로 내보낼 수 있습니다.

[파일 기반 메모리]

파일과 연결된 메모리입니다.

- 실행 파일의 코드
- 공유 라이브러리
- mmap()으로 연결한 파일
- Page Cache

깨끗한 페이지라면 RAM이 부족할 때 그냥 버릴 수 있습니다.

필요해지면 원본 파일에서 다시 읽으면 되기 때문입니다.

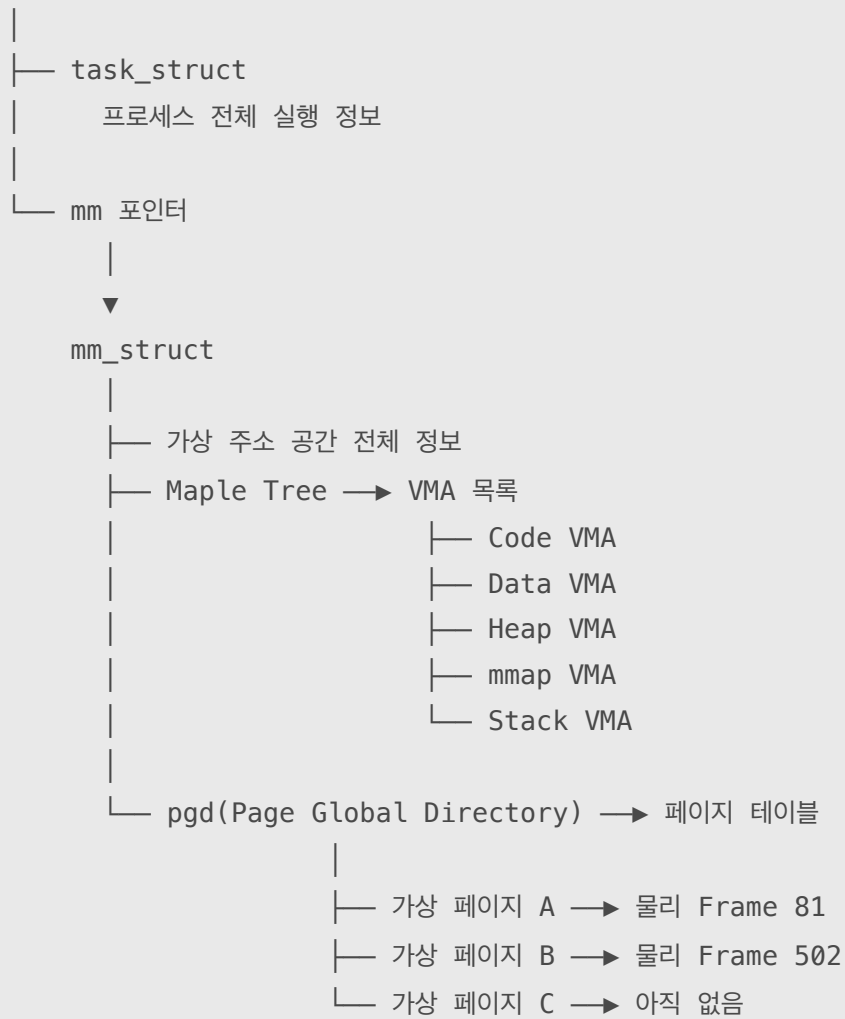
[커널 메모리]

커널도 RAM을 사용합니다.

- task_struct
- mm_struct
- VMA 정보를 담은 Maple Tree
- 페이지 테이블
- 파일시스템의 inode와 dentry
- 소켓과 네트워크 버퍼
- 장치 드라이버 데이터
- Slab/SLUB 객체

[7] 프로세스 하나를 중심으로 본 전체 구조

프로세스 A

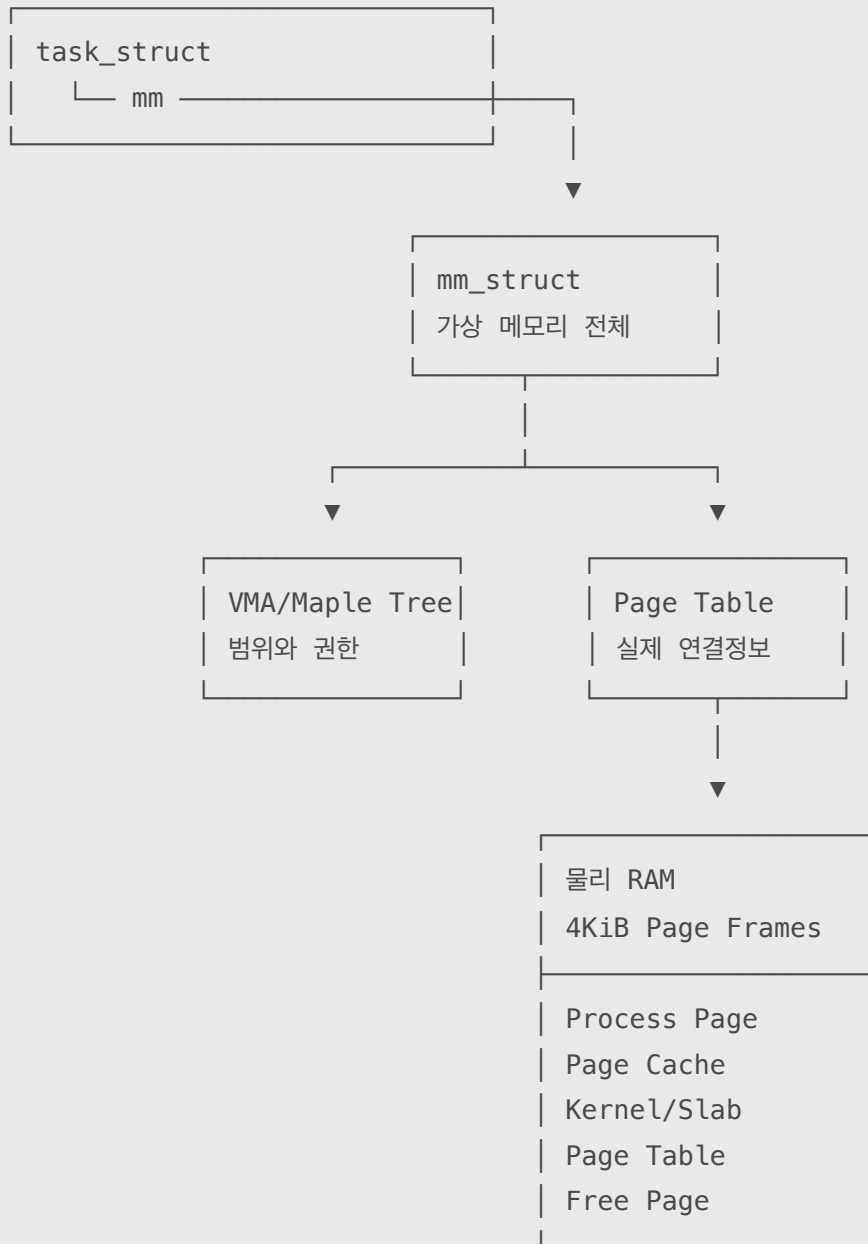


여기서 중요한 차이는 다음과 같습니다.

- VMA: 어떤 가상 주소 범위를 어떤 권한과 용도로 사용할 수 있는지 기록
- 페이지 테이블: 현재 가상 페이지가 실제 어느 물리 프레임과 연결됐는지 기록
- 물리 페이지 프레임: 실제 데이터가 저장되는 RAM 공간

8. 핵심 구조를 한 장으로 정리하면

프로세스



프로세스는 VMA로 구성된 연속적인 가상 주소 공간을 바라보지만,
실제 데이터는 페이지 테이블을 통해 RAM 곳곳의 4KiB 페이지 프레임에 흩어져 저장된다.